

Артур Заплатин

AI / LLM Engineer Intern

Москва · +7 953 820-85-09 · arturzaplatin@gmail.com · [@ArturZaplatin](https://t.me/ArturZaplatin) · github.com/ZaplatinArtur · English B1

КРАТКО ОБО МНЕ

профиль и специализация

Студент 4-го курса УрФУ по направлению «Алгоритмы искусственного интеллекта» и выпускник Летней ML-академии Яндекса. Разрабатываю LLM/VLM-агентов и RAG-системы: строю поиск, управляю использованием контекста и проверяю качество на воспроизводимых экспериментах.

2
завершённых RAG-проекта

91,6%
лучший итоговый accuracy

СТЕК

технологии, с которыми работаю

LLM И АГЕНТЫ Qwen 3.5 9B, LangChain, LangGraph, tool use, PyTorch, Pydantic	RETRIEVAL И RAG FAISS, BM25, E5, RRF, MMR, cross-encoder reranking, Ragas	ДАННЫЕ И BACKEND Python, pandas, SQL, SQLite, FastAPI, PDF/OCR pipelines	ИНСТРУМЕНТЫ Git, Docker, Linux, Hugging Face, OpenRouter, pytest
---	---	--	--

ПРОЕКТЫ

подробно: задача, вклад, действия, результат

01 / ML-Академия Яндекса / 2026

САЙТ ПРОЕКТА
GITHUB

VLM-агент с RAG по турецким учебникам

ЗАДАЧА ПРОЕКТА	Проверить, помогает ли управляемый поиск по турецким учебникам VLM-агенту решать школьные задания лучше, чем та же модель без инструментов.
ЦЕЛЬ ПРОЕКТА	Собрать агента, который сам обращается к RAG, использует только надёжные материалы и не позволяет слабому контексту переписать верный ответ с изображения.
ЧТО ДЕЛАЛ Я	Реализовал основу агентного цикла: вызов RAG, передачу найденных фрагментов в решение, лимит до двух поисков, защиту от повторных запросов и полную историю вызовов с чанками, временем выполнения, ошибками и причиной завершения.
ДЕЙСТВИЯ И ТЕХНОЛОГИИ	Подключил FAISS и фильтры по предмету и классу; добавил порог качества, одну переформулировку запроса и приоритет условия с изображения при конфликте. Реализовал MMR и проверил fetch_k=20, top-5, lambda 0,3 / 0,5 / 0,7 и порядок контекста. Python, Qwen 3.5 9B, PyTorch, LangChain, LangGraph, FAISS, BM25/E5, MMR, Pydantic.

200 УЧЕБНИКОВ корпус проекта	42 981 текстовый индекс	53 329 визуальных страниц	НЕ БОЛЕЕ 2 RAG-вызовов на задачу
--	-----------------------------------	-------------------------------------	--

ПРОБЛЕМЫ И РЕШЕНИЯ	Retrieval возвращал слабые, похожие или противоречивые фрагменты. Слабую выдачу скрывал порог, дубли разводил MMR, число вызовов ограничивал агент, а в спорной ситуации главным источником оставалось изображение задачи.
---------------------------	--

РЕЖИМ	ПРАВИЛЬНО	ACCURACY
Обычный page-RAG	141 / 274	51,5%
Визуальный RAG	171 / 274	62,4%
Агент без инструментов	191 / 274	69,7%
Итоговый пайплайн	251 / 274	91,6%

91,6%
251 правильный ответ из 274
+60 к агенту без инструментов
+110 к обычному page-RAG

РЕЗУЛЬТАТ Итоговый пайплайн решил **251 из 274 задач**: на 60 больше агента без инструментов и на 110 больше обычного page-RAG. Accuracy вырос до **91,6%**.

ОБЩИЙ ВЫВОД **Строгая работа с поиском дала рост до 91,6%.** Итоговый пайплайн сохраняет исходный ответ модели и заменяет его только тогда, когда найденный источник надёжно подтверждает другое решение. Такой контроль добавил 60 правильных ответов к агенту без инструментов.

RAG-система по истории Урала

ЗАДАЧА ПРОЕКТА	Построить RAG по русскоязычным PDF об истории Урала и честно сравнить, какой поиск лучше на текстах с датами, фамилиями, географическими названиями и редкими терминами.
ЦЕЛЬ ПРОЕКТА	Сделать воспроизводимый pipeline от загрузки документов до ответа с источниками и выбрать retrieval-стратегию по измеримым метрикам, а не по нескольким удачным примерам.
ЧТО ДЕЛАЛ Я	Собрал полный цикл: извлечение и очистка текста из PDF, chunking, построение BM25 и FAISS индексов, гибридный поиск через RRF, multi-query, reranking, генерацию через OpenRouter и отдельную оценку retrieval и ответов.
ДЕЙСТВИЯ И ТЕХНОЛОГИИ	Обработал 1 433 страницы и получил 6 300 чанков по 1 200 символов с overlap 200. Сравнил BM25, SentenceTransformers + FAISS, Hybrid RRF, multi-query и cross-encoder reranker на 25 вопросах при top_k=5. Считал Precision@5, Recall@5, Hit@5, MRR, answer F1, groundedness и Ragas sample. Python, pypdf, pdfplumber, FAISS, BM25, SentenceTransformers, RRF, OpenRouter, Ragas.
ПРОБЛЕМЫ И РЕШЕНИЯ	На исторических текстах BM25 оказался сильным благодаря точным совпадениям, а чистый semantic search терял часть имён и дат. Общий reranker тоже ухудшил метрики. Я сохранил эти отрицательные результаты, выбрал hybrid как устойчивую базу и зафиксировал следующий шаг: multilingual embeddings и reranker.

PDF →		pages + chunks →	BM25 + FAISS →	RRF / rerank →	OpenRouter →	evaluation
КОРПУС 1 433 страницы PDF		РАЗБИЕНИЕ 6 300 чанков	HYBRID RRF Recall@5 0,8200 MRR 0,7867		EVALUATION 25 вопросов, top-5	
МЕТОД		PRECISION@5	RECALL@5	HIT@5	MRR	
BM25		0,5033	0,7933	0,9200	0,7333	
Semantic FAISS		0,3267	0,7467	0,8000	0,6280	
Hybrid RRF		0,4393	0,8200	0,9200	0,7867	
Multi-query		0,4933	0,6600	0,7600	0,6600	
Hybrid + reranker		0,3233	0,5933	0,6400	0,5033	

Retrieval evaluation: 25 вопросов, top_k=5. Hybrid дал лучший Recall@5 и MRR при Hit@5 0,92; BM25 сохранил лучшую точность выдачи.

РЕЗУЛЬТАТ	Hybrid RRF дал лучший Recall@5 0,8200 и MRR 0,7867 при Hit@5 0,9200. Эксперимент также показал границы усложнения: multi-query и общий reranker на этом корпусе не улучшили базу.
ОБЩИЙ ВЫВОД	Для русскоязычного исторического корпуса наиболее практичным оказался Hybrid RRF : BM25 удерживает точные имена и даты, а FAISS добавляет смысловые совпадения. При этом усложнение пайплайна само по себе не гарантирует рост: каждый reranker и query-expansion нужно проверять отдельной абляцией.

ОБРАЗОВАНИЕ И ОБУЧЕНИЕ

фундамент и практические программы

2023-2027, ОЧНО Уральский федеральный университет ИРИТ-РТФ, 09.03.01 «Алгоритмы искусственного интеллекта»	Летняя ML-академия Яндекса, выпускник, 2026 Студкемп Яндекса, «Алиса и умные устройства» Karpov.courses, ML Engineer, MLOps, SQL, 2024-2025
--	---

ЧТО ИЩУ

стажировка или junior-позиция

Хочу развиваться в задачах **LLM/VLM agents, RAG, retrieval и evaluation**. Особенно интересны команды, где можно не только собрать прототип, но и измерить, почему он работает, где ошибается и как безопасно использовать найденный контекст.